Slice C — Supabase Sessions (DB + RLS + Client)

Owner: Malek (Bassel support) · Branch: `feature/supabase-sessions` · Merge order: first (C → B → A)

Goal: stand up the Supabase project, create the `sessions` table with Row Level Security, add the mobile client (anon key), and write one `sessions` row per play-through. This is the "confirm a `sessions` row was created" half of the Prototype 1 Definition of Done. No Supabase Auth (ADR-004); no secrets in the app (ADR-003).

---

1. Design exploration

Decision 1 — migrations as code vs. dashboard clicks

Option	Verdict
Click tables together in the Supabase dashboard	Rejected — not reviewable, not reproducible
`supabase/migrations/*.sql` via the Supabase CLI	**Chosen** — matches `docs/data-model.md` ("migrations live in `supabase/migrations/`"); every schema change is a reviewed PR including its RLS policy

The repo already has `supabase/config.toml`, so the project is CLI-shaped. Slice C fills in the first migration.

Decision 2 — RLS policy (the key call)

`docs/data-model.md` is explicit: enable RLS on every table; allow demo `insert` / `select` with the anon key while documenting that this is demo-grade, not HIPAA-grade.

Option	Security	Effort	Verdict for P1
Anon `insert` + `select` allowed (data-model starter)	Demo-appropriate; documented limitation	Low	**Chosen for P1**
JWT with `session_id` claim minted by an Edge Function, policies bound to claim	Stronger	High (needs a function)	**Deferred** (data-model "optional hardening (later)")

We ship the documented starter policy and leave a clearly-marked TODO + the hardening path, exactly as the data-model doc prescribes. Reviewers must see the "demo-appropriate, tighten before any non-course audience" note in the migration.

Decision 3 — who generates the `sessions.id`

DB default `gen_random_uuid()` (server-authoritative) and return it via `.select()`. The app stores the returned id in session state for later gameplay rows (P2). Avoids client/DB id drift.

Decision 4 — what goes in `fhir_snapshot`

`docs/data-model.md` allows an optional cached bundle. For P1, **store the mapped `GameContext`, not the raw FHIR bundle** — it's smaller, already PHI-minimized, and matches what later phases use. Raw bundles are never persisted.

---

2. Files this slice adds

```
maggie/
├── supabase/
│   └── migrations/
│       └── 0001_sessions.sql        # table + RLS (one migration, table+policy together)
└── apps/mobile/src/
    ├── services/supabase.ts         # createClient from anon key on Constants.extra
    └── features/session/sessionService.ts  # createSession(), keeps active sessionId
```

Plus a one-line wire-up: after a successful Epic connect + first snapshot load, call `createSession()`. In P1 this can also be triggered from the Patient Snapshot screen so the demo shows "session created."

---

3. The migration — `0001_sessions.sql`

Mirrors the schema and starter policy in `docs/data-model.md` (table + RLS in the same migration).

```
-- supabase/migrations/0001_sessions.sql
create table public.sessions (
  id              uuid primary key default gen_random_uuid(),
  epic_patient_id text not null,
  fhir_snapshot   jsonb,                       -- mapped GameContext (PHI-minimized), optional
  started_at      timestamptz not null default now(),
  ended_at        timestamptz
);

alter table public.sessions enable row level security;

-- ⚠️ DEMO-APPROPRIATE POLICY (course capstone only).
-- Allows anon insert/select. NOT HIPAA-grade. Tighten before any audience beyond the
-- course demo (see docs/data-model.md "optional hardening": JWT session_id claim).
create policy "sessions_anon_insert"
  on public.sessions for insert to anon with check (true);

create policy "sessions_anon_select"
  on public.sessions for select to anon using (true);
```

> Keep the warning comment in the file — it is part of the deliverable and what reviewers check for.

Applying it

```
# one-time, from repo root
supabase login
supabase link --project-ref <your-project-ref>
supabase db push            # applies migrations to the linked project
```

Owner: Malek (Backend/Data Lead). All schema changes go through PR review with the RLS policy in the same migration (data-model.md §"Migration ownership").

---

4. `supabase.ts` — the client

```
import 'react-native-url-polyfill/auto';        // only if a URL error appears at runtime
import { createClient } from '@supabase/supabase-js';
import Constants from 'expo-constants';

const extra = Constants.expoConfig?.extra ?? {};
const url = String(extra.supabaseUrl ?? '');
const anon = String(extra.supabaseAnonKey ?? '');

export const supabase = createClient(url, anon, {
  auth: { persistSession: false },  // ADR-004: no Supabase Auth in v1
});

export const supabaseConfigured = Boolean(url && anon);
```

- Reads `supabaseUrl` / `supabaseAnonKey` from `Constants.extra` — already wired in `app.config.ts` and already probed by the current `App.tsx`'s `envConfigured()`.
- **Anon key only.** The service role key never appears in the app (it belongs to Edge Functions in later phases).

---

5. `sessionService.ts` — create & track a session

```
import { supabase } from '../../services/supabase';
import type { GameContext } from '../../domain/gameContext';

let activeSessionId: string | null = null;

export async function createSession(epicPatientId: string, ctx?: GameContext) {
  const { data, error } = await supabase
    .from('sessions')
    .insert({ epic_patient_id: epicPatientId, fhir_snapshot: ctx ?? null })
    .select('id')
    .single();
  if (error) throw error;
  activeSessionId = data.id;
  return data.id;            // store for P2 gameplay rows
}

export const getActiveSessionId = () => activeSessionId;
```

- `epic_patient_id` comes from Slice A's `AuthContext.patientId`.
- `ctx` is the mapped `GameContext` from Slice B (PHI-minimized) — never the raw bundle.
- In a mock-only run (no Epic yet) you may pass a literal `'SYNTHETIC_SANDBOX_DEMO'` patient id so the "row created" demo works before Slice A lands — but mark such rows clearly.

---

6. Wiring it into the flow

Minimal P1 trigger: once the Patient Snapshot has a `GameContext` and the user is connected, create the session.

```
useEffect(() => {
  if (auth.status === 'connected' && auth.patientId && ctx.source === 'EPIC_SANDBOX') {
    createSession(auth.patientId, ctx).catch(/* show friendly card, never log raw */);
  }
}, [auth.status, ctx.source]);
```

For the demo, you can also expose a tiny "session created ✓ (id …)" line on the snapshot or summary screen so reviewers can see the Definition of Done met. Verify in the Supabase dashboard → Table editor → `sessions`, or:

```
select id, epic_patient_id, started_at from public.sessions order by started_at desc limit 5;
```

---

7. Guardrails / review checklist (Slice C)

- [ ] RLS is **enabled** and the migration includes policies (table + policy in one file).
- [ ] The "demo-appropriate, tighten later" warning comment is present.
- [ ] Mobile uses the **anon** key only; no service role key anywhere in `apps/mobile`.

- [ ] `fhir_snapshot` stores the mapped `GameContext`, never a raw FHIR bundle, never real PHI.
- [ ] `persistSession: false` (no Supabase Auth, ADR-004).
- [ ] No Supabase URL/key hardcoded — read from `Constants.extra` / `.env`.
- [ ] `.env` not committed.

---

8. Commit sequence (feature/supabase-sessions)

This slice has no UI dependency, so it can merge first and unblock "session row created" early.

#	Commit message	Contents	Push?
1	`chore(db): supabase migration scaffold`	confirm `supabase/` layout, `migrations/` dir	Push → open **draft PR**
2	`feat(db): sessions table + RLS (demo policy)`	`0001_sessions.sql` with warning comment	Push
3	`chore(mobile): add @supabase/supabase-js`	dependency + lockfile	Push
4	`feat(services): supabase anon client`	`supabase.ts`	Push
5	`feat(session): createSession + active id`	`sessionService.ts`	Push
6	`feat: create session row after connect`	wire-up in snapshot, demo confirmation line	Push
7	`docs(db): note RLS limitation + verification query`	docs (Summer)	Push → **ready for review**

Push commit 1 immediately (draft PR). **Request review** after commit 7. Because nothing here imports Slice A/B code paths required to compile, commits 1–5 can merge to `develop` ahead of the other slices.

---

9. Definition of done (Slice C)

- `supabase db push` applies `0001_sessions.sql`; RLS is on with the documented demo policy.
- The app, using only the anon key, inserts a `sessions` row and gets back its `id`.
- A reviewer can see the new row in the dashboard / via the verification query.
- No service role key, no raw FHIR, no PHI in the table; limitation documented.
- Provides `getActiveSessionId()` that Prototype 2's `quizzes` / `quiz_responses` will reference.
